# CS 64 HANDOUT #3

## Memory Access in MIPS Assembly Programming

Prof. Ziad Matni

*Read this handout **before** coming to class. In lecture, I will expand a lot on the concepts in this document, and give lots of examples. So, these handouts are a good explanation of what I'll be talking about in class, but they are not good substitutes for missing the lecture!* ☺

In this reading, we will discuss and clarify:

- CPU Memory vs. CPU Registers
- CPU Memory General Organization
- Setting up CPU Memory Values in **spim**
- Accessing Memory using **lw** and **sw** Instructions
- Application Example: Integer Array
- The Rules of Memory Addressing in MIPS

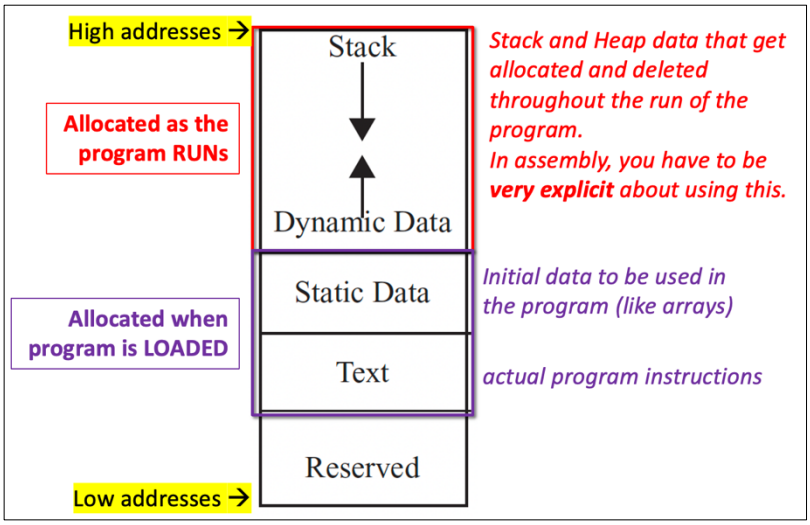
### 1. Is CPU Memory the Same as CPU Registers?

No! (short answer)

Registers are limited in scope and size: there are only 32 of them in a MIPS CPU and each can ONLY hold 32 bits (i.e., 4 Bytes). That's a grand total of 32 x 4 Bytes = 128 Bytes. Not all registers can be used to hold generalized programming data – some of them are specialized for specific use (`$zero`, `$sp`, etc…). Registers are used for much more temporary purposes than CPU memory.

CPU memory is much, much vaster! In MIPS, we have about 2 GB (Gibi-Bytes or ~$2^{31}$ bytes – see section **6** for an explanation of this). Even if some of that memory is not directly usable by the programmer ("Reserved"), it is still a lot of storage. CPU memory is often referred to as CPU RAM (random-access memory). While we can use CPU memory for temporary purposes inside a program, the idea is that it gets accessed a lot less often than registers.

## 2. CPU Memory General Organization




MIPS CPU memory is organized as shown in the diagram. The "Text" part of memory (starting address = 0x0040 0000, see MIPS Reference Sheet for details) is where program code, i.e., instructions, get loaded and executed from.

The "Static Data" part (starting address = 0x1000 0000) is where we may want to have some of a program's initialized data like strings or int arrays. It's also where we want to place values that stay the same throughout the program (constants, global variables) or otherwise remain unchanged (static variables).

The use of the Stack and the Dynamic Data (i.e., Heap) is left to be explicitly used in a program and we'll learn more about this use in a later lesson.

## 3. Setting up CPU Memory Values in **spim**

When using the assembler tool **spim**, setting up CPU memory values before program execution happens using the **.data** directive. Place this before the **.text** directive, which is where your program instructions go. For our course, please know how use **.data** to define byte values, word values, string values, and reserved space:

```
.data
var1:   .byte 9         # declare a single byte with decimal value 9
var2:   .word 94 33     # declare 2 32-bit words (4 bytes) w/ decimal value 94, 33
str:   .asciiz "Hi!"    # declare a null-terminated string
arr:   .space 16        # reserve 16 consecutive bytes of space
```

The **.word** and **.space** examples are the C/C++ equivalent of:

```
int var2[2] = {94, 33};
int arr[4];             // since 16 bytes is 4 integers (word-size)
```

As with C/C++, **.word** places values in contiguous memory spaces.

Here's a short example of some assembly programming that uses **.data** values:

```
.data
      name: .asciiz "Jimbo Jones is "
      yold: .asciiz " years old."
      newline: .asciiz "\n"
.text
main:
      li $v0, 4
      la $a0, name   # la = load memory address
      syscall
      li $v0, 1
      li $a0, 15
      syscall
      li $v0, 4
      la $a0, yold
      syscall
      la $a0, newline
      syscall
      li $v0, 10
      syscall
```

This prints out "Jimbo Jones is 15 years old.", followed by a newline.

### 4. Accessing Memory using `lw` and `sw`

**`lw`** and **`sw`** are core MIPS instructions (and both are I-Types) that allow the programmer to access CPU memory. **Load-word** (**`lw`**) dereferences a value *from* a memory address and places that value in a register, while **store-word** (**`sw`**) places a word value from a register *into* a memory address.

Syntactically, they have to be used as follows (showing with examples):
**`lw $rt, offset($rs)`**, for example: **`lw $t0, 0($t5)`**, where we go to the memory address in register `$t5`, offset that by `0`, and then *dereference* that address (*read* the value). The resultant value gets placed in register **`$t0.`**

**`sw $rt, offset($rs)`**, for example: **`sw $t1, 4($t6)`**, where we go to the memory address in register `$t6`, offset that by 4 (i.e., address `$t6 + 4`), and then *write* the value in register `$t1` at that address location**.**

For example, consider the following program snippet:

```
.data
     num0: .word 42
     num1: .space 4
.text
main:
     la $t0, num0          # t0=num0 (an address value)
     la $t1, num1          # t1=num1 (an address value)
     lw $t2, 0($t0)        # t2=*t0 = 42

     addi $v0, $t2, 42     # v0 = 42+42 = 84
     sw $v0, 0($t1)        # *t1=84, i.e., *num1=84
```

Note how **lw** is used with **la** (load-address): we first acquire the memory address of **num0**, by placing it in the register **$t0**, and then we dereference that address and place the value in **$t2**. Also note how **sw** is used with **la** (load-address): again, we first get the memory address of **num1** and then we use **sw** to write the value 84 to that memory location.

## 5. Application: An Integer Array

Consider the following C/C++ program:

```
int myArray[] = {5, 32, 87, 95, 286, 386};
static int myArrayLength = 6;

for (int i = 0; i < myArrayLength; i++) {
     cout << myArray[i] << endl;
}
```

Note that, in assembly, we would have to first pre-define the integer array **myArray**, the static integer **myArrayLength**, and the **newline** character in the **.data** section.

One way to write this program in MIPS assembly is as follows:

1. Setup the `.data` section.
2. In the `.text` section, under main: label, first initialize the necessary registers:
   a. `$t0 = i = 0`
   b. `$t1 = myArrayLength`
   c. `$t2 = *($t1) = 6`
   d. `$t3 = myArray`
3. Setup the for-loop:
   a. If **i < $t2**, then engage with the loop and otherwise, exit the program
   b. Get the next word value in the array: ***($t3 + 4*i)**
   c. Print that value, print a newline character
   d. Increment **i++**
   e. Repeat step (3)(a)

A more efficient method will be shown next, but let's first demonstrate how this works (see next page).

```
.data
     newline:         .asciiz "\n"
     myArray:         .word 5 32 87 95 286 386
     myArrayLength:   .word 6
.text
main:
     # let $t0 be index i; initialize i to 0
     li $t0, 0

     # get myArrayLength, put dereferenced result in $t2
     la $t1, myArrayLength
     lw $t2, 0($t1)

     # get the base address of myArray
     la $t3, myArray
loop:
     # if i < myArrayLength (i.e., 6), set $t9=1
     # leave the loop, if not true (i.e. $t9=0)
     slt $t9, $t0, $t2
     beq $t9, $zero, exit

     # get the next value in the array to print out.
     # This is going to be the array address + (index * 4).
     # We multiply by four to index bytes as opposed to words
     # because addressing in MIPS happens in bytes (more on this later)
     sll $t4, $t0, 2
     addu $t4, $t4, $t3

     # print out myArray[i], with a newline
     lw $a0, 0($t4)
     li $v0, 1
     syscall
     la $a0, newline
     li $v0, 4
     syscall

     # increment index
     addi $t0, $t0, 1
     # restart loop
     j loop

exit:
     # exit the program
     li $v0, 10
     syscall
```

A much more efficient way to translate this code is to take advantage of the presence of pointer-language, which is naturally done in MIPS assembly (**la** with the use of **lw**, for instance).

Consider if we re-wrote the original C/C++ code as:

```
int myArray[] = {5, 32, 87, 95, 286, 386};
static int myArrayLength = 6;
int p*;

for (p = myArray; p < myArray + myArrayLength; p++) {
     cout << *p << endl;
}
```

Here, we can have the loop work with actual memory address instead of an index. We can build the loop this way and then dereference the pointer only when we want to print it out. This is, in fact, also the sort of thing an optimized compiler can do with a high-level language program. Using the same **.data** section, the **.text** section can now be rewritten like this:

```
main:
     la $t0, myArray            # This is variable p
     la $t1, myArrayLength
     lw $t2, 0($t1)             # t2 = 6: no. of elements
     sll $t3, $t2, 2            # t3 = 6 * 4 = 24: no. of bytes in array
# t4 = myArray address + myArrayLength.
# This is the address of the END of the array (in bytes):
     addu $t4, $t0, $t3

loop:
     # see if current p < address of END of array
     # if so, then exit the loop
     slt $t2, $t0, $t4     # it's ok to reuse $t2!!
     beq $t2, $zero, exit

# otherwise dereference p and print it out with a newline:
     lw $a0, 0($t0)  # a0 = value at current p
     li $v0, 1
     syscall
     la $a0, newline
     li $v0, 4
     syscall

     # increment pointer p by 4 (to get to the next word)
     addiu $t0, $t0, 4
     j loop
exit:
     li $v0, 10
     syscall
```

## 6. Rules of Memory Addressing in MIPS

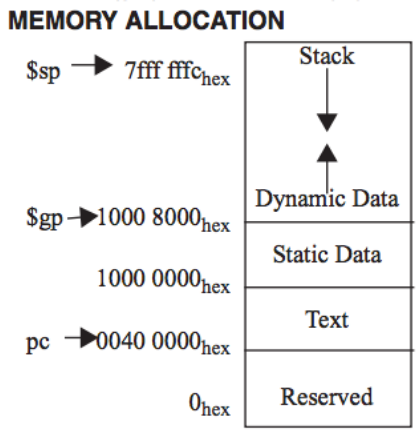
In the majority of CPUs (including MIPS), memory is addressed in **BYTES**. Recall that, in MIPS, memory addresses are made up of 32 bits each, i.e., 4 Bytes.

So, if 2 integers (word-size) are placed next to each other in CPU memory, and the first one has a memory address **0x1000 1000** (as an example), then the other integer has an address of **0x1000 1004**. Moreover, if we are somehow placing a character (1 Byte) in memory contiguous to an integer (4 Bytes), and if the address of the character is **0x1000 1000** (again, as an example), then the integer's address is **0x1000 1004**, and NOT **0x1000 1001**. This is because CPU memory is organized with an alignment rule to best work with word-size values (refer to your notes from CMPSC 32 on memory padding).

As you know by now, a memory address is always a positive number and there are 32-bits to each memory address. ***But***, remember that memory addressing is done in ***Bytes*** (*not in bits! not in words!*), therefore each memory address refers to one Byte of memory.

In a MIPS CPU, we thus have: *$2^{32}$ addresses, meaning $2^{32}$ Bytes = ($2^2$ x $2^{30}$) Bytes =* ***4 Gibi-Bytes**** * In CS/CE, we often use kibi/Mibi/Gibi instead kilo/Mega/Giga because we're dealing with numbers that are *powers of 2*, not powers of 10. Thus, while a **kilo** is $10^3$ = 1000, a ***kibi*** is $2^{10}$ or, 1024. Similarly, a Mibi is $2^{20}$ (i.e., a $2^{10}$ kibi, just like a Mega is a thousand kilos) and a Gibi is $2^{30}$, and so on. When you buy a computer at a retailer, they may claim things like: "*This computer comes with 8 GB of RAM! Oh my!*" and pronounce it "8 Giga-Bytes", what they *really* mean is that it comes with 8 *Gibi*-Bytes, but for marketing purposes (and because no one outside of CS/CE uses words like "Gibi"!!), they'll say "Giga".

Not all of the MIPS CPU memory's 4 Gibi-Bytes is useable or addressable by the programmer because a little more than half of it is "reserved" for use by the Operating System. So, the actual usable memory in MIPS is more like 2 Gibi-Bytes.




If you look at the diagram shown on the left (it's from the MIPS Reference Sheet), you'll see starting addresses listed for all the memory sections. Note that the highest address is **0x7FFF FFFC** and not **0xFFFF FFFF**. *Why is that?!*

First of all, the high addresses of **0xFFFF FFFF** to **0x8000 0000** are reserved for the OS – this comprises one half of all the MIPS CPU memory. As for the top useable address being **0x7FFF FFFC**, let's take a look at the *addressing convention* that MIPS uses: **Big Endian**, and compare it to its counterpart, **Little Endian**.

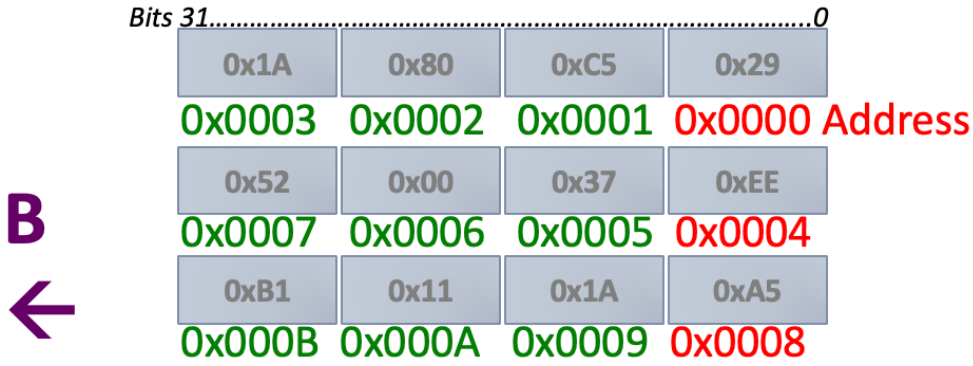
## 7. Addressing Conventions: Big Endian vs. Little Endian

As already hinted at, even though memory bytes are all placed next to each other and thus have contiguous addresses, we like to group them in 4-byte groups. Many popular structures or collections of bits in CPU memory take up 4-bytes, like integers, or instructions. This sets an *alignment restriction* in how we store some data structures in memory.

When we address bytes in memory, say words (i.e., 4 bytes), we can either address the entire 4 words by the address of the most-significant byte first, i.e., the "big end":


**Big Endian Convention for Addressing**

Or we can address them by the least-significant byte first, i.e., the "little end":




**Little Endian Convention for Addressing**

The first scheme (A) is called **Big Endian** convention, and the other (B) is called **Little Endian**.

Most CPUs can handle either convention, but it is best for a CPU to pick one as a default and use that. There are no real advantages for a CPU to use one convention over the other.
MIPS typically (by default) uses Big Endian. Many Intel x86 CPUs go with Little Endian.

---

*Next up is a new reading and lesson on how instructions actually get **disassembled** for MIPS CPUs into the 32 bits of actual 1s and 0s.*
*Following that, we'll get into **programming with functions** in MIPS assembly.*